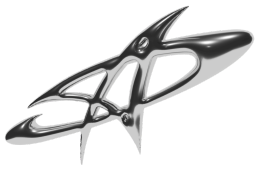
Week 02 · Sequence Data, Tokenizer & Embedding

시퀀스 데이터, 토크나이저, 임베딩 그리고 RNN/LSTM

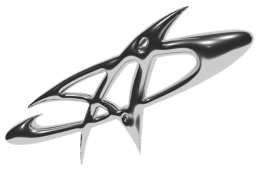
RNN과 LSTM은 그림으로 감만 잡고 넘어갑니다. 오늘의 중심은 시퀀스를 어떻게 숫자로 바꾸는지(토크나이저·임베딩)이고, 진짜 목표는 “왜 Attention이 필요해졌는가”라는 문제의식을 갖는 것입니다.



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

이론 세션

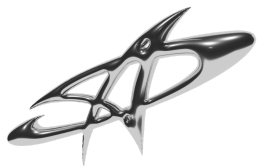
시퀀스 데이터,
그리고 숫자로 표현하는 법



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

“순서” 자체가 정보인 데이터

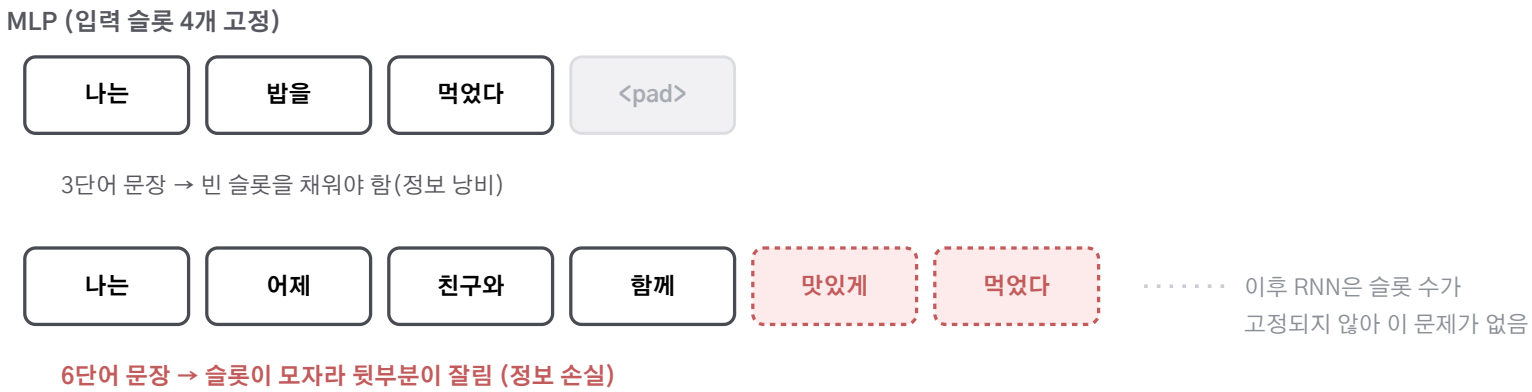
- 문장(“나는 밥을 먹었다” ≠ “밥을 나는 먹었다”), 시계열(주가), 음성, DNA 서열 등
- 1주차에서 쓴 MLP는 입력 크기가 **고정**이어야 하고, 위치 정보를 자동으로 이해하지 못함
- 문장마다 길이가 다르고(”나는 밥을 먹었다” vs ”나는 어제 친구와 함께 밥을 맛있게 먹었다”), 순서를 바꾸면 의미가 완전히 달라짐
- → **가변 길이**를 다루면서 **순서 정보**를 반영하는 구조가 필요 — 이것이 RNN의 출발점



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

MLP가 시퀀스에 약한 이유

입력 슬롯이 고정되어 있다면

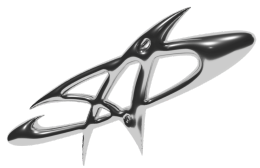


MLP는 입력 벡터의 크기를 미리 정해야 하지만, 문장은 길이가 제각각입니다 — RNN은 “같은 셀을 반복 사용”해 이 문제를 피합니다.

이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

텍스트를 어떻게 숫자로 바꿀 것인가

- 모델은 텍스트를 그대로 이해하지 못하고, **숫자(정수 ID)**만 입력으로 받을 수 있음
- **토크나이저(tokenizer)** = 문장을 “토큰”이라는 단위로 쪼갠 뒤, 각 토큰을 정수 ID로 매핑하는 도구
- 등장 가능한 모든 토큰을 모아놓은 사전을 **vocab(어휘 사전)**이라 하고, 토큰↔ID 매핑을 **stoi / itos**라 부름
- 문장의 시작·끝, 배치 길이 맞추기, 미등록 단어를 표시하는 **특수 토큰**(<sos>, <eos>, <pad>, <unk>)도 vocab에 함께 등록



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

토크나이저의 역할

원문 → 토큰 → 정수 ID



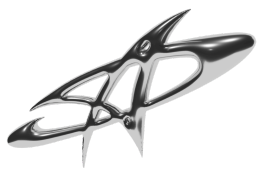
텍스트 → 토큰 → 정수 ID 순서로 바뀝니다. 이 정수들이 잠시 후 배울 Embedding을 거쳐 의미를 담은 벡터가 됩니다.

이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

토큰화 단위, 무엇을 기준으로 쪼갤까

문자 vs 단어 vs 서브워드

단위	VOCAB 크기	시퀀스 길이	OOV(미등록 단어) 처리	예시
문자(char)	작음 (수십~수백 개)	매우 김 (문장 길이 = 글자 수)	거의 없음 — 웬만한 글자는 vocab에 이미 있음	“먹었다” → 먹 / 었 / 다
단어(word)	매우 큼 (수만~수십만 개)	짧음 (문장 길이 = 단어 수)	학습 때 못 본 단어는 **<unk>**로 처리해야 함	“먹었다” → 먹었다 (통째로 1개)
서브워드(subword)	중간 (보통 1만~5만, 직접 설정)	중간	희귀 단어도 아는 조각들로 쪼개 OOV를 최소화	“먹었다” → 먹었 + 다 (자주 쓰는 조각 단위)



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

자주 붙어 나오는 조각을 반복적으로 합친다

1단계 — 모든 글자를 낱개로 분리

n	e	w	e	s	t

(newest 예시)

2단계 — 말뭉치에서 가장 흔한 쌍 'e+s'를 하나로 병합

n	e	w	es	t

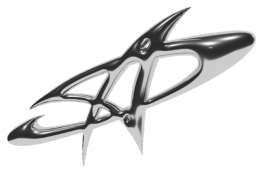
← newest+widest에서 9회 등장

3단계 — 'es+t'를 다시 병합

n	e	w	est

← 새 토큰 'est'로 vocab에 등록

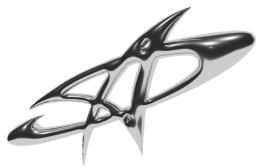
이 병합을 정해둔 vocab 크기만큼 반복합니다 — 자주 등장하는 조각일수록 큰 단위의 토큰이 되고, 희귀한 조합은 글자 단위로 남아 처음 보는 단어도 아는 조각들의 조합으로 표현할 수 있습니다.



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

토큰 ID에 “의미”를 담기

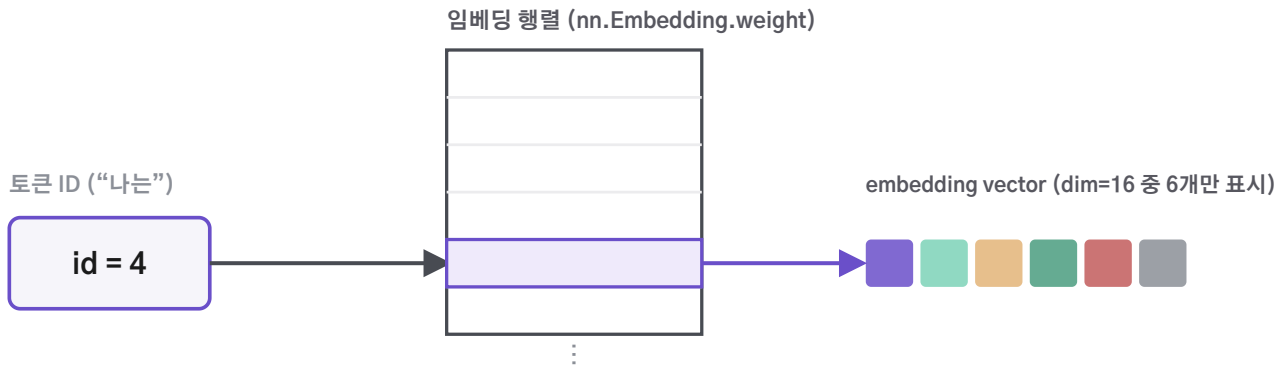
- 토큰 ID는 그저 순번일 뿐 — ID 7과 8이 붙어있다고 의미가 비슷한 건 아님
- One-hot으로 표현하면 vocab 크기만큼 차원이 필요하고, 대부분이 0인 **희소(sparse)** 벡터가 됨
- **Embedding**은 각 토큰 ID를 고정 크기의 **실수 벡터(dense vector)**로 바꿔주는, 학습되는 lookup table
- PyTorch에서는 **nn.Embedding(vocab_size, embed_dim)** 하나로 구현 — 내부는 [vocab_size, embed_dim] 크기의 행렬



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

Embedding lookup

토큰 ID → 행렬의 한 행(row) → 벡터



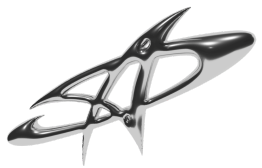
id=4(“나는”)가 들어오면 임베딩 행렬에서 그 행(row)을 그대로 꺼내옵니다 — 이 행 벡터가 학습을 통해 점점 “나는”의 의미를 담게 됩니다.

이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

Embedding이 학습하는 것

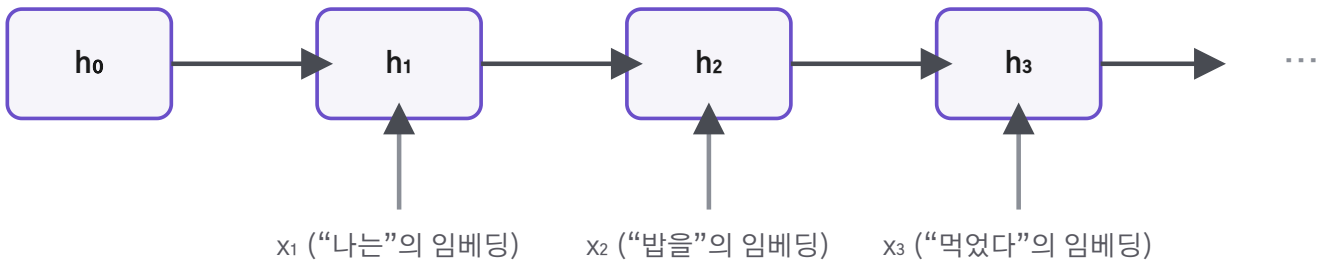
비슷한 맥락의 토큰은 벡터도 가까워진다

- 학습 초기에는 랜덤 벡터지만, 학습이 진행될수록 **비슷한 문맥에서 쓰이는 토큰끼리 벡터 공간에서 가까워짐**
- 고전적 예시: vector(king) − vector(man) + vector(woman) ≈ vector(queen) — 의미 관계가 벡터의 방향·거리로 나타남
- nn.Embedding의 가중치(weight)는 **모델의 다른 파라미터와 함께 역전파로 학습**됨 — 별도로 미리 학습시키는 과정이 필요 없음
- 이 벡터들이 이후 RNN/LSTM, 그리고 다음 주 배울 Attention에 들어가는 **입력**이 됨



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

같은 셀(cell)을 시간축으로 펼치면

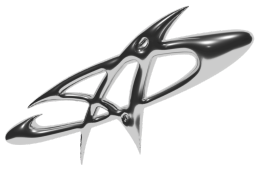


같은 파라미터(같은 셀)를 매 타임스텝마다 재사용하면서, 이전 은닉 상태와 현재 입력(단어의 임베딩 벡터)을 함께 넣어 다음 은닉 상태를 만든다.

이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

문장이 길어질수록, 초반 정보는 계속 곱해지고 곱해지며 흐려진다.
이것이 기울기 소실(vanishing gradient) 문제다.

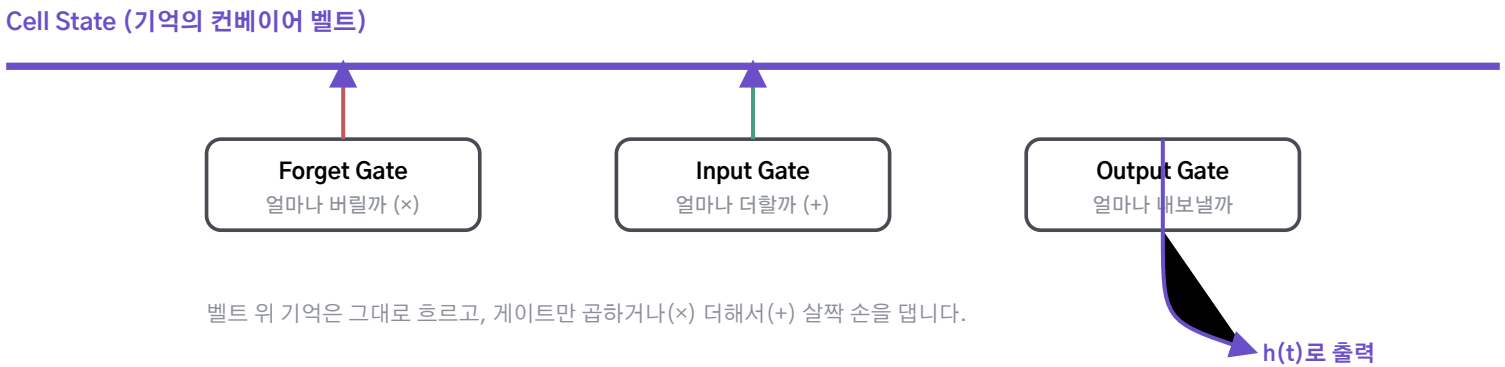
“그 남자는 … (100단어) … 그는 배가 고팠다”에서
“그 남자”를 50번째 스텝까지 기억하기가 점점 어려워진다.



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

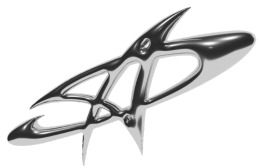
LSTM 게이트, 그림으로

“컨베이어 벨트” 위의 세 개의 밸브



벨트 위 기억은 그대로 흐르고, 게이트만 곱하거나(×) 더해서(+) 살짝 손을 댑니다.

Forget Gate가 지우고(×), Input Gate가 새로 더하고(+), Output Gate가 지금 내보낼 만큼만 걸러냅니다 — 벨트(cell state) 자체는 끊기지 않고 이어집니다. (게이트 수식은 암기 대상이 아닙니다 — “게이트로 기억을 관리한다”는 아이디어만 기억하면 충분합니다.)



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

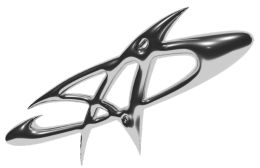
RNN/LSTM vs Transformer, 근본적인 차이

RNN / LSTM

- 한 스텝씩 **순차적으로만** 처리 (병렬화 어려움)
- 먼 거리의 단어 관계는 여러 스텝을 거쳐야 전달됨
- 문장이 길어질수록 앞부분 정보가 옅어짐

Transformer (Self-Attention)

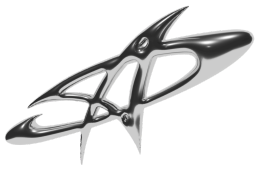
- 모든 단어를 **한 번에, 병렬로** 처리
- 거리에 상관없이 모든 단어 쌍을 직접 연결
- “멀리 있어도 관련 있으면 바로 본다”



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

RNN/LSTM은 “기억을 순서대로 전달”하려 애썼지만,
Transformer는 아예 다른 질문을 던진다.

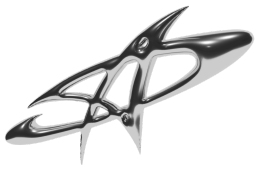
“각 단어가, 문장 속 다른 모든 단어를 한 번에 직접 보게 하면 어떨까?”
이 질문의 답이 다음 주부터 배울 Self-Attention이다.



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

코딩 세션

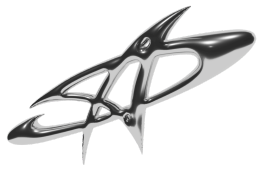
토크나이저 & 임베딩 직접 구현하기



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

토크나이저 → 임베딩, 순서대로 손으로 구현

- ① **예외처리 없는** 문자 단위 vocab — 모르는 글자가 들어오면 무슨 일이 벌어지는지 직접 확인
- ② **예외처리 있는** 단어 단위 vocab — <unk>로 안전하게 처리하기
- ③ **nn.Embedding**으로 토큰 ID를 벡터로 바꾸기
- ④ 문장을 전처리(encode+padding)해서 실제로 embedding에 흘려보내고 shape 확인하기



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

① 예외처리가 없는 VOCAB

문자 단위 vocab 만들기 + encode/decode

01_char_vocab.py

```
text = "나는 밥을 먹었다. 너는 밥을 먹었니?"

# 등장하는 모든 문자를 모아 vocab 생성 (예외처리 없음)
chars = sorted(set(text))
stoi = {ch: i for i, ch in enumerate(chars)}
itos = {i: ch for ch, i in stoi.items()}

def encode(s):
    return [stoi[ch] for ch in s]

def decode(ids):
    return "".join(itos[i] for i in ids)

print(encode("나는"))   # [3, 5]

# 학습 데이터에 없던 문자가 들어오면?
encode("나도 그렇게 생각해.")
# KeyError: '도' — 예외처리가 없어서 그대로 프로그램이 멈춘다
```



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

② 예외처리가 있는 VOCAB

단어 단위 vocab + <unk>로 안전하게

02_word_vocab_with_unk.py

```
sentences = ["나는 밥을 먹었다", "너는 밥을 먹었니"]

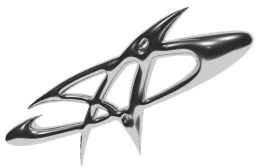
# 특수 토큰: 패딩, 문장 시작/끝, 미등록 단어
SPECIALS = ["<pad>", "<sos>", "<eos>", "<unk>"]

words = sorted(set(w for s in sentences for w in s.split()))
vocab = SPECIALS + words
stoi = {w: i for i, w in enumerate(vocab)}

def encode(sentence):
    # 사전에 없는 단어는 <unk>로 대체 (예외처리)
    ids = [stoi.get(w, stoi["<unk>"]) for w in sentence.split()]
    return [stoi["<sos>"]] + ids + [stoi["<eos>"]]

print(encode("나는 밥을 먹었다"))   # [1, 4, 8, 7, 2]
```

<pad>는 배치 안에서 길이를 맞출 때, **<sos>/<eos>**는 문장의 시작과 끝을, **<unk>**는 처음 보는 단어를 표시할 때 사용합니다.



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

③ EMBEDDING

nn.Embedding은 텐서만 입력으로 받는다

03_embedding_basics.py

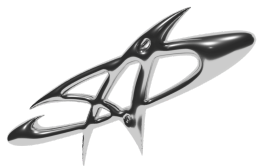
```
import torch.nn as nn

vocab_size, embed_dim = 50, 16
embedding = nn.Embedding(vocab_size, embed_dim)

text = ["안녕", "나는 오늘 밥을 먹었어."]

# 리스트를 그대로 넣으면 에러 — embedding도 하나의 layer이기 때문
embedding(text)
# TypeError: argument 'indices' must be Tensor, not list
```

nn.Embedding도 nn.Module이라 **텐서(Tensor)** 입력만 받습니다 — 텍스트를 넣기 전에 encode → padding → tensor 변환이 필요합니다.



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

④ 전처리 → Embedding

encode + padding 후 embedding에 흘려보내기

04_preprocessing_and_embedding.py

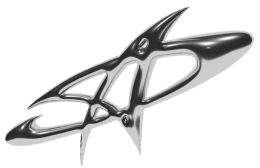
```python
import torch

def preprocessing(text, padding_size=10, padding_token=0):
    input_tokens = []
    for sentence in text:
        encoded = encode(sentence)
        padded = encoded + [padding_token] * (padding_size - len(encoded))
        input_tokens.append(padded)
    return torch.tensor(input_tokens)

text = ["안녕", "나는 오늘 밥을 먹었어."]
input_data = preprocessing(text)
print(input_data.shape)        # [2, 10] — [batch, seq_len]

embedded = embedding(input_data)
print(embedded.shape)          # [2, 10, 16] — [batch, seq_len, embed_dim]
```

raw text → **preprocessing(encode+padding)** → **embedding** 순서로 흐릅니다. 배치 안 문장 길이를 맞추기 위해 짧은 문장은 <pad>(=0)로 채웁니다.



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

nn.Embedding 주의사항

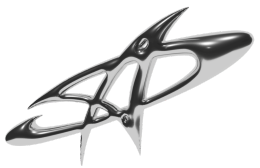
인덱스 범위를 벗어나면 에러

05_embedding_index_error.py

```
embedding = nn.Embedding(50, 16)

# vocab_size(50)는 "행의 개수"일 뿐, 인덱스로는 못 씀
embedding(torch.tensor([50]))
# IndexError: index out of range in self
```

유효한 인덱스는 **0~49** — vocab_size는 항상 실제 등장하는 토큰 수(+특수 토큰)보다 크거나 같게 잡아야 합니다. **projector.tensorflow.org**에서 학습된 임베딩 공간을 직접 탐색해볼 수 있습니다.



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

2주차 정리

오늘 배운 것 & 다음 주 예고

오늘 배운 것

→시퀀스 데이터 = 순서 자체가 정보 → 가변 길이 + 순서를 다루는 구조가 필요

→토큰화 = 텍스트를 문자/단어/서브워드 단위로 쪼개 정수 ID로 바꾸는 과정

→BPE 같은 서브워드 방식이 Vocab 크기와 OOV 문제를 동시에 완화

→Embedding = 토큰 ID를 의미 있는 dense 벡터로 바꾸는, 학습되는 lookup table

→RNN/LSTM은 순차 처리 구조라 병렬화가 어렵고 장기 의존성에 약함

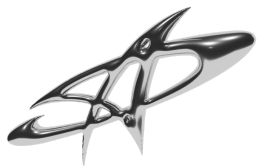
3주차 예고

→Query, Key, Value — Attention의 핵심 직관

→Scaled Dot-Product Attention 수식

→코딩: Attention을 PyTorch로 처음부터 직접 구현

→attention score를 눈으로 시각화



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지